

SHEIN 服装 PLM

+ Multi-Agent AI 智能体

完整业务解决方案 -- 从趋势采集到热销复盘，全链路自动化
Multi-Agent MCP 工具协议 三层 Memory LangGraph PLM 深度耦合

方案设计：马永 | 2026 年 6 月

执行摘要

核心业务目标

用 Multi-Agent 智能体 打通 PLM 全流程，完成 趋势采集 to 选品定款 to 款式生成 to 成本校验 to 热销复盘 to 数据回流迭代 全自动化。

量化指标

指标	现状	目标
选品耗时	3-5天/款	2小时内
人工重复工作	趋势采集/成本核算/录入	替代70%
爆款率	依赖个人经验	AI辅助+30%
PLM数据一致性	人工录入易出错	零差错自动同步

LangGraph 多智能体编排 + MCP 标准化工具协议 + 三层 Memory + PLM 深度耦合

业务完整故事：真实选品全链路

角色

选品设计师 | 供应链专员 | 运营复盘专员 | AI 产品经理

9 步闭环流程

Step 1

设计师上传社媒穿搭图，输入："生成3款2026夏季辣妹牛仔短裤，匹配近3个月热销版型，成本≤35元"

Step 2

总管Agent解析需求，拆解为4个子任务，分发给4个专业Agent并行执行

Step 3

趋势Agent：MCP爬取Instagram/Pinterest/TikTok潮流+向量库RAG检索流行元素

Step 4

PLM数据Agent：MCP读取历史款销量/退货/面料BOM/工厂成本

Step 5

多模态绘图Agent：生成效果图+拆解版型/面料/辅料

Step 6

供应链校验Agent：核算成本，拦截超预算款式，返回修改方案

Step 7

汇总Agent：整合输出选品报告+款式图，一键写入PLM生成工艺单

Step 8

复盘Agent：上新2周后拉取销售数据，总结热销/滞销特征，生成训练集

Step 9

模型进化：训练数据入库，增量微调行业大模型

行业痛点与为什么是AI Agent

6大痛点

痛点	损失
趋势采集依赖人工刷社媒 1-2天	错过潮流窗口期
销售/成本/BOM数据分散多个系统	选品缺乏全局数据
从概念到效果图 3-5天	SKU受限
款式确定后才核算成本	返工浪费排期
复盘靠Excel，经验无法结构化	每次从头来
设计师离职带走爆款经验	组织记忆缺失

为什么不是普通 workflow？

RAG Chatbot 不够：只能回答问题，不能"做事情"（查库/调模型/写PLM）

拖拽 workflow 不够：线性流程无法处理多约束并行决策

Multi-Agent + MCP 是答案：每个Agent专注一个领域，MCP标准化工具调用，总管Agent动态编排——这正是Anthropic/OpenAI 2026主打范式

四层技术架构总览

第1层：前端业务应用层

Go 仪表盘
JD要求的dashboard
React 前端
PLM内嵌面板
Celery 任务队列
异步批量任务
RBAC 权限
角色数据隔离

第2层：Multi-Agent 编排层（核心）

LangGraph
Agent图编排
AutoGen

多Agent对话
5类Agent
总管/趋势/数据/绘图/校验
三层Memory
工作/会话/长期

第3层：MCP 标准化工具层

数据库MCP
PLM/销售/BOM
多模态MCP
文生图/CLIP
爬虫MCP
社媒数据抓取
PLM同步MCP
自动写回款式

第4层：底层数据&模型算力层

MySQL/PostgreSQL
PLM业务主数据
Milvus/Chroma
多模态向量检索
DeepSeek/Claude
推理+规划
数据治理管线
训练集自动生成

全业务流程泳道图（分角色+步骤+对应技术）

泳道1：设计师（业务发起端）

步骤	操作	技术实现
1	在PLM内嵌AI面板输入选品需求 + 上传参考穿搭图	Go Web仪表盘 + 多模态图片预处理 (resize/format/EXIF清洗)
2	等待Agent输出选品方案，人工审核调整参数（成本、风格）	会话记忆缓存本次需求上下文，无需重复输入
3	确认方案，一键写入PLM系统	MCP PLM同步服务 to 创建款式档案 + 生成BOM工艺单

泳道2：Multi-Agent编排中心（核心流程）

步骤	操作	技术实现
1	规划Agent拆解任务，分发4个子Agent并行执行	LangGraph多智能体调度 + Plan任务拆解逻辑 + JSON Schema结构化输出
2	各Agent通过MCP工具层调用外部资源，并行返回结果	MCP标准化工具调用 + Redis会话共享上下文
3	汇总Agent校验所有数据，执行业务硬规则	业务规则引擎 + LLM结果校验 + 自动纠错重跑
4	生成完整选品报告+款式效果图，写入长期记忆库	Mem0长期记忆 + 多模态PDF生成pipeline

泳道3：MCP工具执行层（数据读写/模型调用）

步骤	操作	技术实现
1	MCP数据库服务读取PLM历史款销量、物料成本	MCP数据库网关 + SQL脱敏审计 + 多库异构数据对齐
2	MCP多模态服务生成新款效果图、拆解款式属性	多模态模型推理流水线 + CLIP图像特征提取
3	MCP同步服务将最终方案回写PLM系统	PLM开放API + 数据字段映射中间件

泳道4：底层数据&模型迭代（后置复盘）

步骤	操作	技术实现
1	上新2周后，复盘Agent定时拉取销售数据	Celery定时任务 + 数据湖批量读取
2	自动清洗热销/滞销案例，生成训练数据集入库	数据治理脚本 + 向量库新增行业样本
3	增量微调行业大模型，优化选品泛化能力	RAG知识库更新 + 模型微调管线（LoRA/QLoRA）

JD职责 to 技术方案逐条落地对照

#	JD职责描述	本方案落地	技术关键词
1	服装行业Agent智能产品，选品定款、款式分析、热销总结落地	Multi-Agent分工体系（5类Agent）+ MCP数据库工具 + 向量RAG潮流检索，完整覆盖选品全场景	Multi-Agent MCP RAG
2	跨部门研发/市场/销售协同	PLM系统双向同步MCP工具 + 统一RBAC权限体系 + 全流程操作日志审计，各部门数据互通	RBAC AuditLog MCP
3	多模态模型落地Pipeline，Plan/Tool Usage/Memory/Multi-Agent迭代范式	四层架构全覆盖：LangGraph规划引擎 + MCP标准化工具协议 + 三层记忆机制 + 多智能体协作框架	LangGraph MCP Memory
4	解决大模型推理、长上下文、复杂指令、代码能力；采集提升泛化的数据	分层记忆解决上下文丢失 + MCP沙箱安全执行SQL/代码 + 后置数据治理管线自动生成行业微调数据集	Zep/Mem0 DataGovernance
5	完整产品生命周期管理	Go前后台产品面板 + 任务调度 + 线上运营监控 + 版本迭代 + 业务指标看板（爆款率、研发工时节省）	Go Dashboard

为什么不是简单拖拽工作流能实现

核心差异对比

维度	普通拖拽工作流（n8n/Zapier/Make）	本企业级Multi-Agent方案
流程形态	单条线性流程，A to B to C顺序执行	并行+分支+回退：多Agent同时执行，成本不通过自动返回修改
权限管控	无细粒度权限	RBAC角色隔离：设计师看不到成本，供应链看不到竞品数据
多系统适配	需要逐个系统写Connector	MCP标准协议统一管理所有数据库/模型/系统接入
记忆管理	无记忆，每次重新来	三层记忆：工作/会话/长期，历史方案可检索复用
智能决策	硬编码规则	LLM动态规划：根据需求复杂度自动调整Agent协作策略
PLM耦合度	表层调用API	深度耦合：自动创建款式档案、生成BOM工艺单、同步物料表
持续进化	搭完即结束	数据迭代管线：自动生成训练集 to 增量微调 to 模型越来越准

结论：拖拽工作流只能做「单人临时工具」，本方案是「企业级AI选品平台」--从数据采集到模型迭代的完整闭环。核心差异在于 MCP标准化工具协议 + Multi-Agent动态编排 + 三层记忆 + PLM深度耦合 四个维度的企业级能力。

部署架构与基础设施

Docker Compose 开发环境

```
# docker-compose.yml
version: '3.8'
services:
  # === Agent 编排服务 ===
  agent-orchestrator:
    build: ./agent-orchestrator
    ports: ["8000:8000"]
    environment:
      - LLM_API_KEY=${DEEPSEEK_API_KEY}
      - LLM_BASE_URL=https://api.deepseek.com/v1
      - REDIS_URL=redis://redis:6379
      - MILVUS_HOST=milvus
    depends_on: [redis, milvus]

  # === MCP 数据库网关 ===
  mcp-db-gateway:
    build: ./mcp-servers/db-gateway
    ports: ["8001:8001"]
    environment:
      - PLM_DB_URL=postgresql://plm_user:pass@plm-db:5432/plm
      - SALES_DB_URL=mysql://sales_user:pass@sales-db:3306/sales
      - AUDIT_ENABLED=true

  # === MCP 多模态服务 ===
  mcp-multimodal:
    build: ./mcp-servers/multimodal
    ports: ["8002:8002"]
    environment:
      - SD_API_URL=http://stable-diffusion:7860
      - CLIP_MODEL=ViT-L/14
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities: [gpu]

  # === MCP 爬虫服务 ===
  mcp-crawler:
    build: ./mcp-servers/crawler
    ports: ["8003:8003"]
    environment:
      - PLAYWRIGHT_BROWSERS_PATH=/ms-playwright
      - PROXY_POOL_URL=http://proxy-pool:5010

  # === Redis 会话缓存 ===
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  # === Milvus 向量数据库 ===
  milvus:
    image: milvusdb/milvus:v2.3.0
    ports: ["19530:19530", "9091:9091"]
    environment:
      - ETCD_ENDPOINTS=etcd:2379
      - MINIO_ADDRESS=minio:9000

  # === PLM 主数据库 ===
  plm-db:
    image: postgres:16
    ports: ["5432:5432"] # 内部，不对外暴露
    environment:
      - POSTGRES_DB=plm
      - POSTGRES_USER=plm_user
```

```
- POSTGRES_PASSWORD=${PLM_DB_PASSWORD}
volumes:
- plm_data:/var/lib/postgresql/data

volumes:
  plm_data:
```

Kubernetes 生产环境架构

组件	K8s资源	副本数	资源
Agent Orchestrator	Deployment + HPA	3-10	2CPU 4Gi
MCP DB Gateway	Deployment	2-5	1CPU 2Gi
MCP Multimodal	Deployment + GPU NodeSelector	2-4	4CPU 16Gi + A10 GPU
MCP Crawler	Deployment + CronJob	2	2CPU 4Gi
Redis	StatefulSet + Sentinel	3	2CPU 4Gi
Milvus	StatefulSet	3	8CPU 32Gi

监控与可观测性

层面	工具	指标
基础设施	Prometheus + Grafana	CPU/内存/GPU利用率、网络IO
应用层	OpenTelemetry + Jaeger	Agent调用链路追踪、MCP工具耗时
业务层	自研Go Dashboard	选品任务数、爆款率、成本节省
告警	AlertManager + 飞书/钉钉	Agent失败率>5%、GPU OOM、DB连接池耗尽

实施路线图（4个Phase）

Phase 1：基础能力搭建（4周）

周	里程碑	交付物
W1	环境搭建 + 数据接入	Docker Compose开发环境、PLM数据库只读副本、Redis/Milvus部署
W2	MCP工具层上线	数据库MCP Server（SQL查询+脱敏）、多模态MCP Server（文生图API封装）
W3	基础Agent编排	LangGraph单Agent原型：趋势检索Agent跑通端到端流程
W4	Phase 1 Demo	前端AI面板原型 + 单Agent（趋势检索）完整演示

Phase 2：Multi-Agent联调（4周）

周	里程碑	交付物
W5-6	5类Agent全部上线	总管/趋势/数据/绘图/校验Agent独立运行，通过MCP协作
W7	Memory系统	Redis会话记忆 + Mem0长期记忆集成
W8	Phase 2 Demo	完整选品流程Demo：需求 to 多Agent并行 to 汇总报告 to PLM写入

Phase 3：PLM集成与业务闭环（4周）

周	里程碑	交付物
W9-10	PLM双向同步	MCP PLM同步服务：自动创建款式档案、BOM工艺单、物料表
W11	复盘+数据回流	Celery定时任务 + 训练数据集自动生成管线
W12	权限+审计上线	RBAC角色隔离 + 全流程操作日志审计

Phase 4：生产化与持续优化（持续）

里程碑

交付物

性能优化 Agent并发调度优化、模型推理加速 (vLLM)、向量索引剪枝

业务指标 Go Dashboard: 爆款率、研发工时节省、AI选品采纳率

模型迭代 月度增量微调、A/B测试框架、模型效果评估

教程：MCP Server 完整实现（4类服务）

1. 数据库 MCP Server (Python mcp库)

```
from mcp.server import Server
import mcp.types as types
import asyncpg, json

server = Server("plm-db-server")

@server.list_tools()
async def handle_list_tools():
    return [
        types.Tool(
            name="query_hot_styles",
            description="查询热销款式（销量/退货率），成本字段自动脱敏",
            inputSchema={
                "type": "object",
                "properties": {
                    "days": {"type": "integer", "default": 90},
                    "category": {"type": "string", "default": "牛仔"}
                }
            },
        ),
        types.Tool(
            name="write_style_to_plm",
            description="将AI生成款式写入PLM系统，创建款式档案",
            inputSchema={
                "type": "object",
                "properties": {
                    "style_data": {"type": "object"}
                },
                "required": ["style_data"]
            },
        )
    ]

@server.call_tool()
async def handle_call_tool(name, arguments):
    conn = await asyncpg.connect(host="plm-db", database="plm")

    if name == "query_hot_styles":
        rows = await conn.fetch("""
            SELECT style_id, style_name, sales_14d, return_rate,
                   CASE WHEN unit_cost <= $1 THEN true ELSE false END as within_budget
            FROM styles WHERE category = $2
            AND created_at >= NOW() - INTERVAL '1 day' * $3
            ORDER BY sales_14d DESC LIMIT $4
        """, arguments.get("cost_limit", 35), arguments.get("category", "牛仔"),
        arguments.get("days", 90), arguments.get("limit", 10))
        await conn.close()
        return [types.TextContent(type="text", text=json.dumps([dict(r) for r in rows]))]

    elif name == "write_style_to_plm":
        s = arguments["style_data"]
        await conn.execute("""
            INSERT INTO styles (style_name, category, design_images, fabric_bom, unit_cost, created_by)
            VALUES ($1, $2, $3, $4, $5, 'AI_AGENT')
        """, s["name"], s["category"], json.dumps(s["images"]), json.dumps(s["bom"]), s["cost"])
        await conn.close()
        return [types.TextContent(type="text", text='{"status": "created"}')]
```

2. MCP多模态Server

```
@server.list_tools()
async def handle_list_tools():
    return [
```

```

types.Tool(
    name="generate_design_image",
    description="根据描述生成服装款式效果图（文生图）",
    inputSchema={
        "type": "object",
        "properties": {
            "prompt": {"type": "string"},
            "num_images": {"type": "integer", "default": 3}
        },
        "required": ["prompt"]
    }
)

]

@server.call_tool()
async def handle_call_tool(name, arguments):
    if name == "generate_design_image":
        async with httpx.AsyncClient(timeout=120) as client:
            resp = await client.post(
                "http://stable-diffusion:7860/sdapi/v1/txt2img",
                json={
                    "prompt": f"{arguments['prompt']}, fashion design sketch, clean background",
                    "negative_prompt": "blurry, watermark, text",
                    "steps": 30, "cfg_scale": 7, "width": 768, "height": 1024,
                    "batch_size": arguments.get("num_images", 3)
                }
            )
        return [types.TextContent(type="text", text=json.dumps(resp.json()))]

```

3. MCP爬虫Server

```

@server.list_tools()
async def handle_list_tools():
    return [
        types.Tool(
            name="crawl_trend_data",
            description="爬取Instagram/Pinterest/TikTok服装潮流数据",
            inputSchema={
                "type": "object",
                "properties": {
                    "keywords": {"type": "array", "items": {"type": "string"}},
                    "platforms": {"type": "array", "default": ["instagram", "pinterest"]},
                    "max_results": {"type": "integer", "default": 50}
                },
                "required": ["keywords"]
            }
        )
    ]

@server.call_tool()
async def handle_call_tool(name, arguments):
    if name == "crawl_trend_data":
        from playwright.async_api import async_playwright
        results = []
        async with async_playwright() as p:
            browser = await p.chromium.launch(headless=True)
            for kw in arguments["keywords"]:
                page = await browser.new_page()
                await page.goto(f"https://www.pinterest.com/search/pins/?q={kw}")
                await page.wait_for_timeout(3000)
                pins = await page.evaluate("""() => {
                    return [...document.querySelectorAll('[data-test-id="pin"]')]
                        .slice(0, 20).map(p => ({
                            image: p.querySelector('img')?.src,
                            title: p.querySelector('img')?.alt
                        })
                )""")
                results.extend(pins)
            await browser.close()
        return [types.TextContent(type="text", text=json.dumps(results[:50]))]

```

4. MCP PLM同步Server

```

@server.list_tools()
async def handle_list_tools():
    return [
        types.Tool(

```

```

        name="sync_style_to_plm",
        description="将AI选品结果同步写入PLM: 创建款式档案+生成BOM工艺单+物料表",
        inputSchema={
            "type": "object",
            "properties": {
                "style_name": {"type": "string"},
                "category": {"type": "string"},
                "design_images": {"type": "array"},
                "fabric_bom": {"type": "object"},
                "unit_cost": {"type": "number"},
                "tech_pack": {"type": "object"}
            },
            "required": ["style_name", "category"]
        }
    )
]

@server.call_tool()
async def handle_call_tool(name, arguments):
    if name == "sync_style_to_plm":
        async with httpx.AsyncClient() as client:
            # 1. 创建款式档案
            style_resp = await client.post(
                "https://plm-api.internal/api/v1/styles",
                json={"name": arguments["style_name"], "category": arguments["category"],
                    "images": arguments.get("design_images", []), "source": "AI_AGENT"}
            )
            style_id = style_resp.json()["id"]

            # 2. 生成BOM工艺单
            bom_resp = await client.post(
                f"https://plm-api.internal/api/v1/styles/{style_id}/bom",
                json=arguments.get("fabric_bom", {})
            )

            # 3. 同步物料表
            if arguments.get("tech_pack"):
                await client.post(
                    f"https://plm-api.internal/api/v1/styles/{style_id}/techpack",
                    json=arguments["tech_pack"]
                )

            return [types.TextContent(type="text",
                text=json.dumps({"status": "synced", "style_id": style_id}))]

```

教程：Memory 三层存储架构

架构总览

层级	存储	生命周期	用途
工作记忆	Python dict / 内存	单次Agent步骤	临时工具调用结果，步骤完成后销毁
会话记忆	Redis	整轮选品任务	上下文+中间结果，保留7天供回溯
长期记忆	Milvus + Mem0	永久	历史爆款方案/用户偏好/错误案例，供RAG检索和模型微调

核心实现

```

import redis, json
from datetime import timedelta, datetime
from mem0 import Memory

class WorkingMemory:
    """单步骤临时上下文 - Agent调用工具的结果暂存"""
    def __init__(self): self.store = {}
    def set(self, k, v): self.store[k] = v
    def get(self, k): return self.store.get(k)
    def clear(self): self.store = {}

class SessionMemory:
    """会话记忆 - Redis存储，整轮选品任务所有数据"""
    def __init__(self, redis_url="redis://localhost:6379"):
        self.redis = redis.from_url(redis_url)

```

```

def save_session(self, session_id, data, ttl_hours=168):
    self.redis.setex(f"session:{session_id}", timedelta(hours=ttl_hours),
                     json.dumps(data, default=str))

def get_session(self, session_id):
    raw = self.redis.get(f"session:{session_id}")
    return json.loads(raw) if raw else {}

def append_step(self, session_id, step_name, result):
    """追加Agent步骤结果到会话"""
    session = self.get_session(session_id)
    session.setdefault("steps", []).append({
        "step": step_name, "timestamp": str(datetime.now()), "result": result
    })
    self.save_session(session_id, session)

class LongTermMemory:
    """长期事件记忆 - Mem0 + Milvus, 持久化爆款方案和用户偏好"""
    def __init__(self): self.mem0 = Memory()

    def save_design_solution(self, solution, metadata):
        """保存爆款方案: 设计数据 + 销售结果"""
        self.mem0.add(
            messages=[{"role": "assistant", "content": json.dumps(solution, ensure_ascii=False)}],
            user_id=metadata.get("designer_id", "system"),
            metadata={"type": "design_solution", **metadata}
        )

    def search_similar(self, query, top_k=5):
        """检索相似历史方案 (供趋势Agent使用) """
        return self.mem0.search(query=query, limit=top_k)

    def save_error_case(self, error_desc, context):
        """保存模型错误案例 (用于后续微调) """
        self.mem0.add(
            messages=[{"role": "system", "content": error_desc}],
            metadata={"type": "error_case", "context": context}
        )

```

三层记忆在选品流程中的协作

阶段	使用哪层	读写操作
设计师输入需求	会话记忆	写入 session_id to 原始需求文本
趋势Agent爬取数据	工作记忆	暂存爬虫结果 to 传给设计Agent
PLM数据Agent查库	工作记忆	暂存SQL结果 to 传给供应链Agent
汇总Agent生成报告	会话记忆	读取所有步骤结果 to 合并输出
设计师确认方案	长期记忆	写入Mem0: 方案数据+设计偏好
上新2周后复盘	长期记忆	写入销售结果 to 更新向量库
下次选品	长期记忆	RAG检索历史爆款方案作为参考

业务指标与ROI测算

核心业务指标

指标	计算方式	目标值
AI选品采纳率	AI生成方案被设计师确认上新的比例	> 60%
爆款率提升	AI辅助选品 vs 纯人工选品的爆款率对比	+30%
选品耗时缩短	从需求提出到方案确认的平均耗时	3-5天 to 2小时
研发工时节省	设计师/供应链专员在重复工作上节省的时间	70%
成本拦截率	AI在选品阶段拦截超预算款式的比例	> 95%

ROI测算模型

项目	年度金额
节省人力成本	
设计师工时 (20人 x 70% x 15万/年)	210万/年
供应链专员工时 (5人 x 50% x 12万/年)	30万/年
运营复盘工时 (3人 x 80% x 10万/年)	24万/年
减少返工损失	
超预算款式返工 (月均10款 x 5000元/款 x 12月)	60万/年
增收	
爆款率提升带来额外销售额 (假设基数1亿 x 5%增量)	500万/年
年度总收益	824万/年
投入成本	
开发人力 (5人 x 6个月)	150万
GPU算力 (A10 x 2台 x 12月)	36万/年
API调用费用	12万/年
年度总投入	198万/年
ROI	416% (824万 / 198万)

第一年即可收回全部投资，此后每年净收益 626 万+。